

column names. To get those, use `.headers`. You can store meta-commands in a file named `.sqliterc` in your home directory, to automatically use them whenever you run `sqlite3`. To terminate the `sqlite3` program, use the `.quit` or `.exit` meta-command.

The program

The recommended method of accessing SQLite 3 through PHP is using the PHP Data Objects (PDO) library. PDO is a set of classes that enables you to use different relational database management systems interchangeably through a single interface. Covering PDO is outside the scope of this article, but the procedure for using it is similar to most others: connect to the database, execute queries and commands, and close the connection. PDO uses Data Source Names (DSNs) to connect to various database systems. Each DSN has a prefix that specifies which driver is to be used to establish the connection. The general format of a DSN string is `<driver>:<data source>[<other parameters>]`. For example, the driver for SQLite 3 is `sqlite`. To use a database, we must specify the DSN as `sqlite:<database file name>`. Since there is no concept of users in SQLite, we only need to specify the file-name. For other, server-based systems, you can also specify the username and password to be used for the connection.

Assuming that you store the SQLite database `products.sqlite` in a subdirectory named `_private` of the directory where you will store the PHP program, use the following PHP code to open the database:

```
$db = new PDO("sqlite:_private/products.sqlite");
```

This causes `$db` to refer to a connection to the `products.sqlite` database.

 **Tip:** You can also use the DSN `sqlite:memory:` instead of specifying a file name. This will create a database in memory, and not in an on-disk file. Of course, you will have to create tables in it before being able to use it.

To query the database, you can use the `query` method of the PDO class, as shown below:

```
$query = "SELECT name, price FROM products ORDER BY name";
$result = $db->query($query);
foreach ($result as $row) {
    $name = $row['name'];
    $price = $row['price'];
    $html_row = "<<<HTML_ROW
    <tr>
        <td>$name</td>
        <td>$price</td>
    </tr>
```

```
HTML_ROW;

    echo $html_row;
}

$db = null; // close database connection
```

As you may have noticed, to close the database connection, you can simply set the connection variable to `null`.

To insert or update data, we can use the `exec` method of the PDO class.

```
$cmd = "INSERT INTO products(name, price) VALUES('Sprod_name',
$prod_price)";
if ($db->exec($cmd) != 1)
    $err_msg = "There was an error while trying to enter the
product details";
```

The `exec` method returns the number of records affected.

The PDO system may throw exceptions of type `PDOException`. Apart from this, PDO also allows you to create and execute prepared statements, which is useful if you want to execute the same statement repeatedly with varying parameters—but this feature is not used in this article, due to lack of space. You can refer to the PDO documentation, available as part of the PHP documentation, for details.

The complete source of the program is available in the file `"example.php"` [on the LFY website at <http://bit.ly/eYsA5Q>]. Store the `products.sqlite` database file in a subdirectory named `_private` of the directory where you save this program, then run the program. The program is simplistic; there is no proper error handling, and it does not prevent you from erroneously adding the same product repeatedly by refreshing the page after adding a product—but it is only meant to illustrate how to work with SQLite.

Where next?

This concludes our tour of Lighttpd and SQLite—but this was just an introduction! You can explore this combo further, to discover more uses for SQLite, and to enhance the performance and modularisation of your PHP applications. SQLite also provides a C interface, which you can utilise to enjoy other benefits of SQLite apart from Web applications. 

By: Saurav Sengupta

The author is an independent software developer with an interest in both application and systems programming. He has a blog at <http://sauravzone1.blogspot.com/> and you can reach him at sauravzone1@gmail.com.